Deep learning for Bitcoin transaction and address classification: final report

Nov 3, 2026 · @Ren

Summary

Across two weeks and two datasets, deep learning did not beat tree models on Bitcoin classification: what helps is information from the transaction graph, which tree models use just as well once it is given to them as features.

1. **Trees beat neural networks on tabular features.** Elliptic transactions: Random Forest F1 0.826 against 0.678 for the best graph network. BABD-13 addresses: XGBoost macro F1 0.832 (10 classes) against 0.732 for an MLP.
2. **The graph carries real information.** On identical features, graph networks beat the MLP on both datasets: +0.07 to +0.11 F1 on Elliptic, +9 accuracy points on BABD addresses from unseen entities. XGBoost with neighbour-aggregated features is best of all (accuracy 0.945, macro F1 0.821 on unseen entities).
3. **Standard evaluations overstate the scores.** Elliptic's illicit transactions after the dark-market shutdown (step 43) are missed by every model; on BABD-13, a random split puts addresses of one entity on both sides and adds 4 to 11 accuracy points. BABD-13 also contains 1,414 addresses with contradictory labels, and its labels follow H1 clusters.
4. **Dark-web link:** BABD labels almost none of the Venom addresses, but 85 exchange addresses appear among the counterparties of the 8 Venom sets, and one set's onion-site address is very likely inside an exchange's wallet.

Objective and data

The professor's directive was to try deep learning to classify Bitcoin transactions and addresses and to relate it to the dark-web work on the Venom dataset; four data sources were used.

Data	Unit	Size used	Labels	Role
Elliptic (Weber et al. 2019)	transaction	203,769 nodes, 234,355 edges, 46,564 labelled	illicit / licit	transactions, temporal split

Data	Unit	Size used	Labels	Role
BABD-13 (Xiang et al. 2022)	address	544,462 addresses, 148 features	13 behaviour classes	addresses from their own features
BABD transaction window	transaction	13,093,228 transactions, blocks 600,000–605,999	via BABD-13	graph of 94,851 labelled addresses
Venom (Ingemarsson et al. 2025)	onion site	4,382 addresses, 8 sets with full histories	none	dark-web application

Real-CATS was planned as the address dataset, but the Kaggle copy downloaded holds only Ethereum records (77.6 GB uncompressed), so BABD-13 took its place; whether the Ethereum part is of interest is an open question to the professor.

Method

Every neural model was compared with a tree-based baseline on the same split and the same features, and judged on the rare classes rather than accuracy.

- **Elliptic split:** temporal, as in Weber et al.: training on time steps 1–34 (29,894 labelled transactions, 11.6% illicit), test on steps 35–49 (16,670, 6.5% illicit); features standardized on the training period only.
- **BABD-13 split (features only):** stratified random 80/20 (435,569 / 108,893 addresses), as in the paper; the file has no time column. The `SW` column (strong or weak label) was excluded because it describes the label's origin.
- **BABD graph splits:** an **entity split** that puts whole co-spend clusters in training or test (82,269 / 12,578 addresses, 0 shared clusters), and a random split of the same size for comparison.
- **Imbalance:** class weights in every model (`pos_weight` on Elliptic; square-root balanced weights on BABD, since full balancing would weight a Ponzi error about 2,800 times an exchange error).
- **Metrics:** precision, recall, F1 on the illicit class and PR AUC (Elliptic); accuracy, macro F1 and weighted F1 (BABD), plus macro F1 without classes too small to measure.
- **Leakage checks:** no Elliptic edge crosses time steps (0 of 234,355); in the BABD entity split only payment links cross from training to test, and they carry features, never labels.

- **Environment:** Python 3.14, PyTorch 2.15 (CUDA 13.2), PyTorch Geometric 2.8.1, scikit-learn, XGBoost; NVIDIA RTX 3050 6 GB laptop GPU. Neural results are single runs with seed 42 unless three seeds are stated.

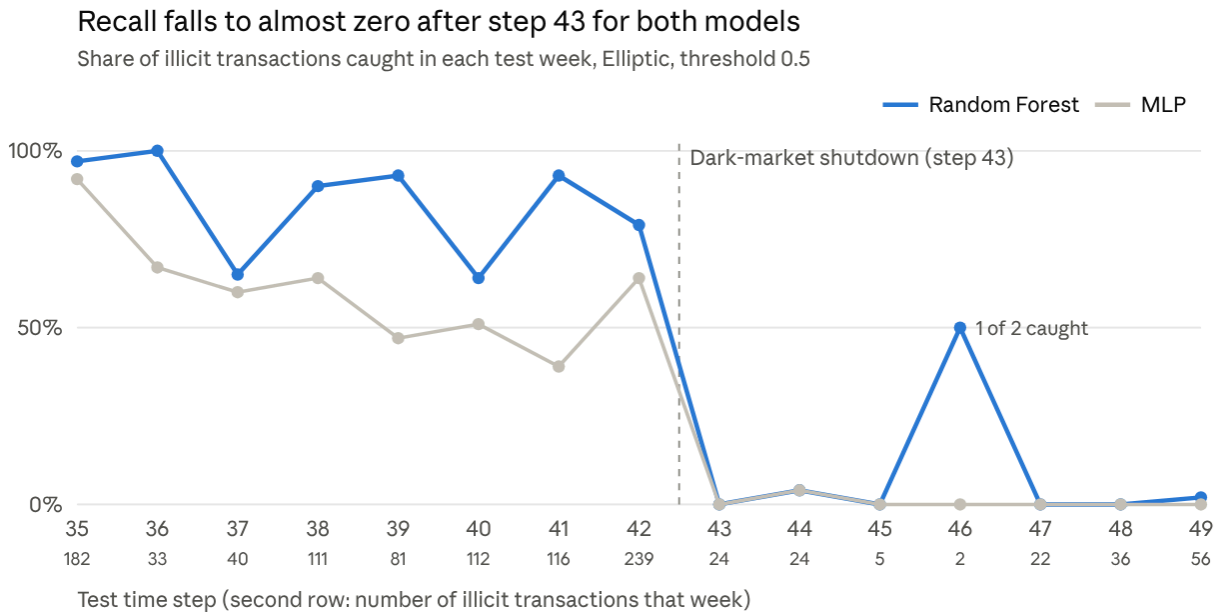
Elliptic: transaction classification

The Random Forest is best by a wide margin (illicit F1 0.826); among neural models, the graph networks beat the MLP by 0.07 to 0.11 F1, mostly by cutting false alarms.

Model	Precision	Recall	F1	PR AUC	False alarms
Random Forest (200 trees)	0.963	0.723	**0.826**	**0.793**	30
GraphSAGE (2 layers, 64 units)	0.806	0.584	0.678	0.640	152
GCN (2 layers, 64 units)	0.788	0.528	0.632	0.649	154
MLP (165-64-32-1)	0.599	0.536	0.565	0.528	389
MLP + validation, early stopping, dropout	0.421	0.513	0.463	0.291	764

Test set: 16,670 transactions, 1,083 illicit; threshold 0.5 except the regularized MLP (0.534, chosen on validation).

- **The GCN reproduces Weber et al.** (F1 0.632 against 0.628).
- **Regularizing the MLP made it worse:** validation on steps 30–34 reached PR AUC 0.864 while test PR AUC fell to 0.291, because the recent past did not predict the later test period.
- *Hypothesis, not tested:* 72 of Elliptic's 165 features already aggregate each transaction's neighbours, so the Random Forest receives much of the graph information through its inputs, as XGBoost did on BABD (below).



Notebook output, test time steps 35–49 · Random Forest and plain MLP

Drift after step 43. Recall on steps 35–42 against 43–49: Random Forest 0.853 against 0.018, GraphSAGE 0.690 against 0.012, MLP 0.633 against 0.006, GCN 0.625 against 0.006. The 169 illicit transactions after the shutdown are missed by every model, as Weber et al. also report.

BABD-13: addresses from their own features

XGBoost reproduces the BABD paper (accuracy 0.970 against 0.967) and beats an MLP on every summary metric.

Model	Accuracy	Macro F1 (13 classes)	Macro F1 (10 classes)	Weighted F1
XGBoost (300 trees, depth 8)	**0.970**	**0.662**	**0.832**	**0.969**
MLP (148-256-128-13, dropout 0.2)	0.921	0.604	0.732	0.922

Stratified random 80/20 split, 108,893 test addresses; the 10-class macro F1 leaves out government blacklist, money laundering and Ponzi, which have 5, 3 and 3 test addresses. The MLP used a sign-log transform, mini-batches of 1,024 and the best of 30 epochs on a 10% validation set.

- **Eight classes are well recognized** (F1 0.86 to 0.99 for XGBoost, 0.65 to 0.96 for the MLP), including darknet market (XGBoost 0.992) and tumbler (0.858).

- **Individual wallet and blackmail are hard for both** (XGBoost 0.07 and 0.59); week 2 found that many of these addresses carry contradictory labels (next section).
- **The MLP's validation score was unstable** (macro F1 between 0.52 and 0.68 from epoch to epoch), because the validation set holds one or two addresses of each tiny class.

BABD-13: label quality and the transaction graph

BABD-13's labels follow co-spend (H1) clusters, so a graph built from co-spending inherits the labels; this shapes how the graph experiment must be evaluated.

The transaction window. The 22.4 GB of JSON in the BABD archive are raw explorer API pages (BTC.com format) for every transaction of blocks 600,000 to 605,999 (late 2019), not ready-made graphs. One scan of the 264,841 files (10.6 minutes) read 13,093,228 transactions and kept the 358,784 that touch a BABD-labelled address; 94,966 labelled addresses (17% of BABD) appear in the window.

Contradictory labels. 1,706 addresses appear more than once in BABD-13 and 1,414 of them carry different labels, mostly blackmail against tumbler (790 addresses) or individual wallet (232). 1,008 extra rows have identical features but a different label, and 528 are exact copies. These 1,414 addresses were removed (540,954 kept). This explains part of the weak blackmail and individual-wallet results above.

The graph. Nodes are the 94,851 labelled addresses of the window with their 148 features; two are linked when they appear in the same transaction, as payer and payee (25,473 links) or as inputs spent together (267,942 co-spend links), 287,596 undirected edges in all. 24.7% of addresses are isolated and the largest component holds 68,991 (73%). Tumblers are the most connected (2.9% isolated), blackmail the least (67.6%).

Labels follow clusters. 98.0% of edges join two addresses of the same class, and 99.3% of the 764 multi-address co-spend clusters carry a single label. BABD takes most labels from WalletExplorer, which labels whole co-spend clusters, so co-spending addresses share a label largely by construction. The largest cluster (37,178 addresses, 39.2%) is an exchange.

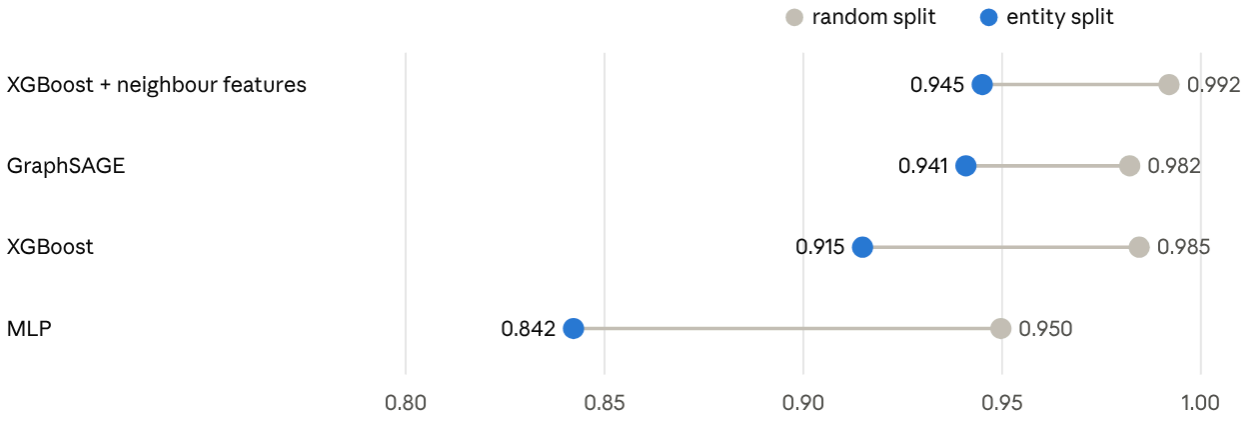
Consequence: an entity split. With a random split, a test address usually has co-spend neighbours in training with the same label, and a graph model could copy it. The entity split puts whole co-spend clusters in training or test (the giant exchange cluster in training), so every test address belongs to an unseen entity. 10 classes with at least 20 addresses were kept (94,847 addresses); the test part holds 12,578 addresses, from 5,166 exchanges down to 4 P2P-infrastructure addresses.

BABD-13 on unseen entities

Neighbour information is what matters on unseen entities: it lifts XGBoost from 0.915 to 0.945 accuracy and the neural model from 0.843 (MLP) to 0.936 (GraphSAGE), and XGBoost with neighbour features stays best on every metric.

On unseen entities every model loses 4 to 11 accuracy points

Test accuracy, BABD-13 addresses in the 6,000-block window, random split against entity split



Axis starts at 0.80. GraphSAGE and MLP: seed 42 (three-seed means on the entity split: 0.936 and 0.843).

Notebook output, 3 November 2026 · 12,578 test addresses per split

Models on the same features and splits: XGBoost on the 148 features; XGBoost on the 148 features plus the mean of each address's neighbours' (log) features and its degree (297 features); an MLP and a two-layer GraphSAGE (128 units, dropout 0.2, square-root class weights, 300 full-batch epochs).

Model	Macro F1, entity	Weighted F1, entity	Macro F1, random	Weighted F1, random
XGBoost + neighbour features	**0.821**	**0.944**	**0.857**	**0.992**
GraphSAGE (3 seeds)	0.691 ± 0.046	0.939 ± 0.003	0.804 (1 seed)	0.983 (1 seed)
XGBoost	0.747	0.915	0.817	0.984
MLP (3 seeds)	0.593 ± 0.014	0.851 ± 0.002	0.699 (1 seed)	0.951 (1 seed)

- **GraphSAGE beats plain XGBoost on the large classes** of the entity split (cyber-security 0.917 against 0.829, gambling 0.959 against 0.920, blackmail 0.495 against 0.424) and loses on the small ones (tumbler 0.357 against 0.688, mining pool 0.42 against 0.83, with 47 and 13 test addresses), hence higher accuracy but lower macro F1.

- **GraphSAGE's macro F1 is unstable across seeds** (0.66 to 0.74), while its accuracy varies by ±0.004.
- **Darknet market generalizes well to new markets** for every model (F1 0.91 to 0.97 on unseen entities).

Venom application

BABD labels say almost nothing about the Venom addresses themselves, but they identify exchange addresses among the counterparties of the 8 Venom sets, and one set's onion-site address is very likely inside an exchange's wallet.

- **Direct overlap:** 2 of the 4,382 Venom addresses are labelled in BABD-13 (one blackmail, one cyber-security service), none of them in the 8 sets. *Hypothesis:* different periods (BABD 2019–2021, Venom April 2024) and sources.
- **No model-based application:** a BABD-trained model needs BABD's 148 features, which cannot be reproduced for Venom addresses without reimplementing the BABD feature code.
- **Counterparties:** 91 of the 58,544 counterparties of the 8 sets are labelled in BABD: 85 exchanges, 4 blackmail, 2 cyber-security. Set 25 accounts for 76 of the exchanges and 358 of its 2,315 transactions include an exchange-labelled address; sets 43, 45 and 667 have none.
- **Set 25 sits inside an exchange wallet (hypothesis, well supported):** exchanges are almost never its direct payment partners (3 payments to an exchange, none from one), but its address is spent together with exchange addresses in 119 transactions, about 37% of the 319 that spend it. Under the common-input heuristic, the same wallet controls co-spent addresses, so the address shown on set 25's onion sites is most likely an exchange deposit address swept with others, the cash-out pattern Lee et al. (2019) describe.

Findings and limitations

The answer to "does deep learning help classify Bitcoin transactions and addresses?" is: not by itself on these data; the transaction graph does, and tree models exploit it as well as graph networks once it is turned into features.

1. **Tree models win on tabular features**, on both datasets, by 0.10 to 0.15 F1 or macro F1.
2. **Graph information is the useful part.** Graph networks beat an MLP with identical features on both datasets; neighbour-aggregated features lift XGBoost by 3 accuracy points on unseen BABD entities, enough to beat GraphSAGE.

3. **Evaluation choices move the numbers more than models do.** Elliptic's post-shutdown period defeats every model; a random split on BABD-13 adds 4 to 11 accuracy points; a validation set from the recent past misled early stopping on Elliptic.
4. **Benchmark labels have flaws:** BABD-13 has 1,414 contradictory addresses and labels that follow H1 clusters, which favours any model that sees co-spend neighbours.
5. **For the dark web,** exchange labels on counterparties were the useful link: one Venom set's address very likely belongs to an exchange's wallet.

Limitations

- One window of 6,000 blocks (late 2019) for the BABD graph, covering 17% of BABD's labelled addresses; links through unlabelled addresses are not in the graph.
- Neural models barely tuned (one architecture each, fixed learning rate); three seeds only on the BABD entity split; other neural results are single runs.
- The entity split approximates entities by co-spend clusters, the giant exchange cluster is always in training, and four classes have 4 to 47 test addresses.
- BABD-13 week-1 results used a random split that may contain the 528 exact duplicate rows on both sides.
- The Venom analysis rests on 91 labelled counterparties, labels from 2019–2021, and the common-input heuristic.

Appendix: code

Cells of `3_deep_learning.ipynb` in the order they ran; data folders are git-ignored.

Week 1: Elliptic and BABD-13 features

A.1 Elliptic: data and temporal split

```
import pandas as pd, numpy as np, torch
from sklearn.preprocessing import StandardScaler

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
DATA = "../elliptic"
feats = pd.read_csv(f"{DATA}/elliptic_txs_features.csv", header=None)
classes = pd.read_csv(f"{DATA}/elliptic_txs_classes.csv")
feats.columns = ["txId", "time_step"] + [f"f{i}" for i in range(165)]
d = feats.merge(classes, on="txId")
d = d[d["class"] != "unknown"].copy()
d["y"] = (d["class"] == "1").astype(int)
tr, te = d[d["time_step"] <= 34], d[d["time_step"] > 34]
xcols = [c for c in d.columns if c.startswith("f")]
scaler = StandardScaler().fit(tr[xcols])
Xtr_t = torch.tensor(scaler.transform(tr[xcols]), dtype=torch.float32,
device=device)
Xte_t = torch.tensor(scaler.transform(te[xcols]), dtype=torch.float32,
device=device)
ytr_t = torch.tensor(tr["y"].values, dtype=torch.float32, device=device)
yte_t = torch.tensor(te["y"].values, dtype=torch.float32, device=device)
```

A.2 Elliptic: MLP and Random Forest

```
import torch.nn as nn
from sklearn.metrics import classification_report, average_precision_score,
confusion_matrix
from sklearn.ensemble import RandomForestClassifier

torch.manual_seed(42)
model = nn.Sequential(nn.Linear(165, 64), nn.ReLU(), nn.Linear(64, 32),
nn.ReLU(), nn.Linear(32, 1)).to(device)
loss_fn = nn.BCEWithLogitsLoss(pos_weight=((ytr_t == 0).sum() / (ytr_t ==
1).sum()).reshape(1))
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
for epoch in range(300):
    model.train(); opt.zero_grad()
    loss = loss_fn(model(Xtr_t).squeeze(1), ytr_t); loss.backward();
opt.step()
model.eval()
with torch.no_grad():
    proba = torch.sigmoid(model(Xte_t).squeeze(1)).cpu().numpy()
y_te = yte_t.cpu().numpy().astype(int)
pred = (proba >= 0.5).astype(int)
```

```
rf = RandomForestClassifier(n_estimators=200, n_jobs=-1, random_state=42)
rf.fit(Xtr_t.cpu().numpy(), tr["y"].values)
rf_proba = rf.predict_proba(Xte_t.cpu().numpy())[:, 1]
rf_pred = (rf_proba >= 0.5).astype(int)
```

A.3 Elliptic: regularized MLP (validation on steps 30–34)

```
import copy
from sklearn.metrics import precision_recall_curve

fit_mask = torch.tensor((tr["time_step"] <= 29).values, device=device)
Xf, yf, Xv, yv = Xtr_t[fit_mask], ytr_t[fit_mask], Xtr_t[~fit_mask],
ytr_t[~fit_mask]
yv_np = yv.cpu().numpy().astype(int)
torch.manual_seed(42)
model = nn.Sequential(nn.Linear(165, 64), nn.ReLU(), nn.Dropout(0.2),
                      nn.Linear(64, 32), nn.ReLU(), nn.Dropout(0.2),
nn.Linear(32, 1)).to(device)
loss_fn = nn.BCEWithLogitsLoss(pos_weight=((yf == 0).sum() / (yf ==
1).sum()).reshape(1))
opt = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)
best_ap, best_state = -1, None
for epoch in range(500):
    model.train(); opt.zero_grad()
    loss = loss_fn(model(Xf).squeeze(1), yf); loss.backward(); opt.step()
    if epoch % 10 == 0:
        model.eval()
        with torch.no_grad():
            pv = torch.sigmoid(model(Xv).squeeze(1)).cpu().numpy()
        ap = average_precision_score(yv_np, pv)
        if ap > best_ap:
            best_ap, best_state = ap, copy.deepcopy(model.state_dict())
model.load_state_dict(best_state); model.eval()
with torch.no_grad():
    pv = torch.sigmoid(model(Xv).squeeze(1)).cpu().numpy()
    proba2 = torch.sigmoid(model(Xte_t).squeeze(1)).cpu().numpy()
p, r, t = precision_recall_curve(yv_np, pv)
thr = t[np.argmax((2 * p * r / (p + r + 1e-12))[:-1])]
pred2 = (proba2 >= thr).astype(int)
```

A.4 Elliptic: graph and GNNs

```
from torch_geometric.data import Data
from torch_geometric.nn import GCNConv, SAGEConv
import torch.nn.functional as F
```

```
edges = pd.read_csv(f"{DATA}/elliptic_txs_edgelist.csv")
all_ = feats.merge(classes, on="txId", how="left")
idx = pd.Series(np.arange(len(all_)), index=all_["txId"])
src, dst = idx[edges["txId1"]].values, idx[edges["txId2"]].values
edge_index = torch.tensor(np.vstack([np.r_[src, dst], np.r_[dst, src]]),
dtype=torch.long)
lab_e = all_["class"].map({"1": 1, "2": 0}).fillna(-1).astype(int).values
step = all_["time_step"].values
scaler_g = StandardScaler().fit(all_.loc[step <= 34, xcols])
x = torch.tensor(scaler_g.transform(all_[xcols]), dtype=torch.float32)
train_mask = torch.tensor((lab_e >= 0) & (step <= 34)).to(device)
test_mask = torch.tensor((lab_e >= 0) & (step > 34)).to(device)
data = Data(x=x, edge_index=edge_index, y=torch.tensor(lab_e)).to(device)

class GNN(nn.Module):
    def __init__(self, conv, in_dim=165, hid=64, p=0.2):
        super().__init__()
        self.c1, self.c2, self.out, self.p = conv(in_dim, hid), conv(hid,
hid), nn.Linear(hid, 1), p
    def forward(self, x, ei):
        h = F.dropout(F.relu(self.c1(x, ei)), self.p, self.training)
        h = F.dropout(F.relu(self.c2(h, ei)), self.p, self.training)
        return self.out(h).squeeze(1)

y_f = data.y.float()
pw = ((data.y[train_mask] == 0).sum() / (data.y[train_mask] ==
1).sum()).float().reshape(1)
def run(conv, epochs=300, lr=5e-3):
    torch.manual_seed(42)
    m = GNN(conv).to(device)
    opt = torch.optim.Adam(m.parameters(), lr=lr, weight_decay=5e-4)
    lf = nn.BCEWithLogitsLoss(pos_weight=pw)
    for ep in range(epochs):
        m.train(); opt.zero_grad()
        loss = lf(m(data.x, data.edge_index)[train_mask], y_f[train_mask]);
loss.backward(); opt.step()
    m.eval()
    with torch.no_grad():
        sc = torch.sigmoid(m(data.x, data.edge_index))
[test_mask].cpu().numpy()
    return sc, (sc >= 0.5).astype(int)
gcn_sc, gcn_pred = run(GCNConv)
sage_sc, sage_pred = run(SAGEConv)
```

A.5 BABD-13: XGBoost and MLP on the features

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score
from xgboost import XGBClassifier
from torch.utils.data import TensorDataset, DataLoader

babd = pd.read_csv(r"data/babd/BABD-13.csv")
feat_cols = [c for c in babd.columns if c not in ("account", "SW", "label")]
Xb, yb = babd[feat_cols].values.astype("float32"), babd["label"].values
Xb_tr, Xb_te, yb_tr, yb_te = train_test_split(Xb, yb, test_size=0.2,
stratify=yb, random_state=42)
xgb = XGBClassifier(n_estimators=300, max_depth=8, learning_rate=0.1,
tree_method="hist", device="cuda", random_state=42)
xgb.fit(Xb_tr, yb_tr); xp = xgb.predict(Xb_te)

def slog(a):
    return np.sign(a) * np.log1p(np.abs(a))
Xb_fit, Xb_val, yb_fit, yb_val = train_test_split(Xb_tr, yb_tr,
test_size=0.1, stratify=yb_tr, random_state=42)
sc_b = StandardScaler().fit(slog(Xb_fit))
to_t = lambda a: torch.tensor(sc_b.transform(slog(a)), dtype=torch.float32)
Xf_b, Xv_b, Xt_b = to_t(Xb_fit), to_t(Xb_val), to_t(Xb_te)
loader = DataLoader(TensorDataset(Xf_b, torch.tensor(yb_fit)),
batch_size=1024, shuffle=True)
counts = np.bincount(yb_fit, minlength=13)
w = (counts.sum() / (13 * counts)) ** 0.5
w = torch.tensor(w / w.mean(), dtype=torch.float32, device=device)
torch.manual_seed(42)
mlp_b = nn.Sequential(nn.Linear(148, 256), nn.ReLU(), nn.Dropout(0.2),
                      nn.Linear(256, 128), nn.ReLU(), nn.Dropout(0.2),
nn.Linear(128, 13)).to(device)
opt = torch.optim.Adam(mlp_b.parameters(), lr=1e-3)
lf = nn.CrossEntropyLoss(weight=w)
def predict(m, X):
    m.eval(); out = []
    with torch.no_grad():
        for i in range(0, len(X), 8192):
            out.append(m(X[i:i + 8192].to(device)).argmax(1).cpu())
    return torch.cat(out).numpy()
best, best_state = -1, None
for epoch in range(30):
    mlp_b.train()
    for xb_, yb_ in loader:
        xb_, yb_ = xb_.to(device), yb_.to(device)
        opt.zero_grad(); loss = lf(mlp_b(xb_), yb_); loss.backward();
opt.step()
```

```
    vf = f1_score(yb_val, predict(mlp_b, Xv_b), average="macro")
    if vf > best:
        best, best_state = vf, copy.deepcopy(mlp_b.state_dict())
mlp_b.load_state_dict(best_state); mp = predict(mlp_b, Xt_b)
```

Week 2: BABD transaction graph and Venom

B.1 Scan of the 6,000 blocks

```
import zipfile, json
from collections import Counter, defaultdict

BABD = r"C:\path\to\BABD.zip"
babd_lab = babd[["account", "label"]]
with zipfile.ZipFile(BABD) as z:
    jsons = [i.filename for i in z.infolist() if
i.filename.endswith(".json")]
by_block = defaultdict(list)
for f in jsons:
    by_block[f.split("/")[2]].append(f)

labels = dict(zip(babd_lab["account"], babd_lab["label"]))
edges = Counter()
act = defaultdict(lambda: [0, 0, 0, None, None])
with zipfile.ZipFile(BABD) as z:
    for b in sorted(by_block, key=int):
        h = int(b)
        for f in by_block[b]:
            for tx in json.load(z.open(f))["data"]["list"]:
                ins = [(a, i.get("prev_value") or 0) for i in tx["inputs"]
for a in (i.get("prev_addresses") or [])]
                outs = [(a, o.get("value") or 0) for o in tx["outputs"] for a
in (o.get("addresses") or [])]
                lab_in = list(dict.fromkeys(a for a, _ in ins if a in
labels))
                lab_out = list(dict.fromkeys(a for a, _ in outs if a in
labels))
                if not lab_in and not lab_out:
                    continue
                for a in set(lab_in) | set(lab_out):
                    s = act[a]; s[0] += 1
                    s[3] = h if s[3] is None else s[3]; s[4] = h
                for a in lab_in:
                    for c in lab_out:
                        if a != c: edges[(a, c, "pay")] += 1
                for a in lab_in[1:]:
```

```
                    edges[(lab_in[0], a, "cospend")] += 1
act_df = pd.DataFrame.from_dict(act, orient="index", columns=["n_tx", "sent",
"received", "first_block", "last_block"])
edges_df = pd.DataFrame([(a, c, k, w) for (a, c, k), w in edges.items()],
columns=["src", "dst", "kind", "n_tx"])
```

B.2 Clean labels, graph, co-spend clusters, splits

```
import networkx as nx
from sklearn.model_selection import StratifiedGroupKFold

per_addr = babd_lab.groupby("account")["label"].agg(["nunique", "first"])
lab_clean = per_addr.loc[per_addr["nunique"] == 1, "first"]
nodes = act_df.index[act_df.index.isin(lab_clean.index)]
e = edges_df[edges_df["src"].isin(nodes) & edges_df["dst"].isin(nodes)]
lab = lab_clean.reindex(nodes)
cospend = e[e["kind"] == "cospend"]
Gc = nx.Graph(); Gc.add_nodes_from(nodes);
Gc.add_edges_from(zip(cospend["src"], cospend["dst"]))
cid = pd.Series({a: i for i, c in enumerate(nx.connected_components(Gc)) for
a in c}).reindex(nodes)

keep_cls = [c for c in range(13) if (lab == c).sum() >= 20]
nodes2 = nodes[lab.isin(keep_cls).values]
y_all, groups = lab.reindex(nodes2), cid.reindex(nodes2)
giant = groups.value_counts().idxmax()
sgkf = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=42)
for k, (tr_e, te_e) in enumerate(sgkf.split(np.zeros(len(nodes2)),
y_all.values, groups.values)):
    if giant not in set(groups.values[te_e]):
        break
tr_r, te_r = train_test_split(np.arange(len(nodes2)), test_size=len(te_e),
stratify=y_all.values, random_state=42)
splits = {"entity": (tr_e, te_e), "random": (tr_r, te_r)}
X = babd.drop_duplicates("account").set_index("account").loc[nodes2,
feat_cols].values.astype("float32")
y = y_all.map({c: i for i, c in enumerate(keep_cls)}).values
```

B.3 XGBoost with neighbour features, MLP and GraphSAGE

```
import scipy.sparse as sp

pos = pd.Series(np.arange(len(nodes2)), index=nodes2)
ee = e[e["src"].isin(pos.index) & e["dst"].isin(pos.index)]
s_, d_ = pos[ee["src"]].values, pos[ee["dst"]].values
ei = np.vstack([np.r_[s_, d_], np.r_[d_, s_]])
```

```
A = sp.csr_matrix((np.ones(ei.shape[1]), (ei[0], ei[1])), shape=(len(nodes2),
len(nodes2)))
deg_n = np.asarray(A.sum(1)).ravel()
X_aug = np.hstack([X, (A @ slog(X)) / np.maximum(deg_n, 1)[:, None], deg_n[:,
None]]).astype("float32")

edge_index_b = torch.tensor(ei, dtype=torch.long, device=device)
y_t = torch.tensor(y, device=device)

class MLP(nn.Module):
    def __init__(self, d, h=128, k=10, p=0.2):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(d, h), nn.ReLU(), nn.Dropout(p),
nn.Linear(h, h), nn.ReLU(), nn.Dropout(p), nn.Linear(h, k))
    def forward(self, x, ei=None):
        return self.net(x)

class SAGE(nn.Module):
    def __init__(self, d, h=128, k=10, p=0.2):
        super().__init__()
        self.c1, self.c2, self.out, self.p = SAGEConv(d, h), SAGEConv(h, h),
nn.Linear(h, k), p
    def forward(self, x, ei):
        h = F.dropout(F.relu(self.c1(x, ei)), self.p, self.training)
        h = F.dropout(F.relu(self.c2(h, ei)), self.p, self.training)
        return self.out(h)

def train_eval(kind, tr, te, epochs=300, lr=5e-3, seed=42):
    sc = StandardScaler().fit(slog(X[tr]))
    xt = torch.tensor(sc.transform(slog(X)), dtype=torch.float32,
device=device)
    cnt = np.bincount(y[tr], minlength=10)
    w = (cnt.sum() / (10 * cnt)) ** 0.5
    w = torch.tensor(w / w.mean(), dtype=torch.float32, device=device)
    torch.manual_seed(seed)
    m = (SAGE(X.shape[1]) if kind == "GraphSAGE" else
MLP(X.shape[1])).to(device)
    opt = torch.optim.Adam(m.parameters(), lr=lr, weight_decay=5e-4)
    lf = nn.CrossEntropyLoss(weight=w)
    trt = torch.tensor(tr, device=device)
    for ep in range(epochs):
        m.train(); opt.zero_grad()
        loss = lf(m(xt, edge_index_b)[trt], y_t[trt]); loss.backward();
```

```
opt.step()
    m.eval()
    with torch.no_grad():
        return m(xt, edge_index_b).argmax(1).cpu().numpy()[te]
```

B.4 Venom counterparties

```
import glob, os, re

lab_any = babd_lab.groupby("account")["label"].agg(lambda s:
tuple(sorted(set(s))))
sel = pd.read_pickle(r"../Venom/data/selected_clusters.pkl")
seed_addrs = set(a for l in sel["addr_key"] for a in l)
rows = []
for path in glob.glob(os.path.join(r"../Venom/data/raw", "c*_a*.json")):
    cid_v = int(re.search(r"c(\d+)_a", os.path.basename(path)).group(1))
    with open(path) as fh:
        for t in json.load(fh):
            for v_ in t["vin"]:
                a = (v_.get("prevout") or {}).get("scriptpubkey_address")
                if a: rows.append((cid_v, t["txid"], "input side", a))
            for o in t["vout"]:
                a = o.get("scriptpubkey_address")
                if a: rows.append((cid_v, t["txid"], "output side", a))
allrows = pd.DataFrame(rows, columns=["set", "txid", "side",
"addr"]).drop_duplicates()
allrows["is_seed"] = allrows["addr"].isin(seed_addrs)
allrows["is_exch"] = allrows["addr"].map(lab_any).apply(lambda t:
isinstance(t, tuple) and 3 in t)
```